

Retrieval-Augmented Generation (RAG): A Conceptual Overview (Demo)

What Problem RAG Solves

Large language models encode knowledge in their parameters at training time, which means that knowledge is frozen at the training cutoff, cannot easily be updated without retraining, and is not traceable back to a specific source. Retrieval-Augmented Generation addresses this by pairing a language model with an external, searchable knowledge base: at query time, relevant documents are retrieved and provided to the model as context, so the model can generate an answer grounded in content it did not need to memorize.

This makes it possible to update the knowledge base independently of the model, attach citations to generated claims, and reduce (though not eliminate) the rate at which the model states unsupported or incorrect facts.

The Retrieval Step

A typical RAG pipeline first splits source documents into chunks, embeds each chunk into a dense vector using an embedding model, and stores those vectors in a vector index such as FAISS or a dedicated vector database. At query time, the user's question is embedded with the same model, and the index returns the chunks whose vectors are most similar to the query vector, usually measured by cosine similarity or inner product on normalized vectors.

Purely semantic (embedding-based) retrieval can miss exact keyword matches — for example, a rare product code or an exact quoted phrase — which is why many production systems combine semantic retrieval with lexical retrieval methods such as BM25, a technique generally referred to as hybrid retrieval.

Chunking Strategy

How a document is split into chunks materially affects retrieval quality. Chunks that are too large dilute the embedding with unrelated content and make citations coarse; chunks that are too small lose surrounding context needed to answer the question. Most practical systems use a chunk size on the order of a few hundred tokens, with a small overlap between consecutive chunks so that information spanning a chunk boundary is not lost entirely from either chunk.

Preserving metadata per chunk — the source document name, page number, and a stable chunk identifier — is essential for producing citations that a user can actually verify against the source material.

Retrieval Improvements

Beyond basic top-k similarity search, several techniques improve retrieval quality: hybrid lexical-semantic search combines exact-match signal with semantic similarity; reranking passes an initial candidate list through a more expensive cross-encoder model to reorder results by relevance; query expansion rewrites or augments the user's query to improve recall; and duplicate-context removal filters near-identical chunks (for example, boilerplate repeated across many documents) so the limited context window is not wasted on redundant text.

Grounding, Refusal, and Prompt Injection

A responsible RAG system should distinguish between what the retrieved evidence actually states (evidence) and any additional reasoning the model layers on top (inference), and should explicitly refuse to answer — rather than guessing — when retrieved evidence does not sufficiently cover the question.

Because retrieved content often comes from external, uncontrolled sources, RAG systems must also treat retrieved text as untrusted data rather than as instructions. Malicious or accidental text embedded in a document (a prompt-injection attempt) should never be able to change the system's behavior — the model should read such text as content to report on, never as a command to follow.